15-213 Recitation
Final Exam Review

Your TAs

Friday, April 24th

Reminders

- **`sfslab`** was due *yesterday*
  - Can use at most 1 grace day
- Code Reviews:
  - **`sfslab`** graded for style, but no meetings
- ***Final Exam Bootcamp***
  - Saturday April 25, 11AM-2PM
  - GHC 4401
- ***Final Exam***
  - Monday April 27, 1-4PM
  - DH 2210, 2315
  - Watch Ed for details!

TA Feedback

- Leave feedback for your TAs!
  - Link for feedback on Ed or click here:

    https://www.ugrad.cs.cmu.edu/ta/S26/feedback/
  - Staff photos are on course website.
- All feedback is welcome!
  - Feel free to rant, or give suggestions.

Agenda

- **Final Exam Resources**
- **Final Exam Practice**
  - *Today:*
    - Assembly
    - Shell & File IO
    - Synchronization & Threading
  - Bootcamp will cover more topics!

Final Exam Resources

Ed #612, #737, #777

- **Lecture/Recitation Slides**
- **Labs:** what are the key concepts you learned?
- **Textbook:** especially useful for second half of the course
- **Written Assignments**
- **Practice Final Exam**
- **Past Final Exams**

Assembly

Assembly

- Common exam questions:
  - Fill in the blanks of a C code snippet given assembly
  - Given assembly, determine the behavior of a function (like in **`bomblab`**!)
- Make sure to brush up on assembly!
  - Addressing modes: e.g., **`D(Rb, Ri, S)`**
  - Control Flow: **`cmp`** and condition codes
  - Calling Conventions
  - Good thing to include on your cheat sheet

Assembly: FITB

S11 Q3

```
typedef struct node {
     void *data;
     struct node *next;
} node_t;

node_t *fun(node_t *n, int f(node_t *)) {
     node_t *a, *b;
     if (_____________________) {
          return NULL;
     }

     a = _____________________;

     if (_____________________) {
          b = _____________________;
          b->data = n->data;
          b->next = _____________________;
          return b;
     }

     return _____________________;
}
```

```
114d: push   %r12
114f: push   %rbp
1150: push   %rbx
1151: mov %rdi,%rbx
1154: test   %rdi,%rdi
1157: je  1192 <fun+0x49>
1159: mov %rsi,%rbp
115c: mov 0x8(%rdi),%rdi
1160: call   1149 <fun>
1165: mov %rax,%r12
1168: mov %rbx,%rdi
116b: call   *%rbp
116d: test   %eax,%eax
116f: jg  1179 <fun+0x30>
1171: mov %r12,%rax
1174: pop %rbx
1175: pop %rbp
1176: pop %r12
1178: ret
1179: mov $0x10,%edi
117e: call   1050 <malloc@plt>
1183: mov (%rbx),%rdx
1186: mov %rdx,(%rax)
1189: mov %r12,0x8(%rax)
118d: mov %rax,%r12
1190: jmp 1171 <fun+0x28>
1192: mov %rdi,%r12
1195: jmp 1171 <fun+0x28>
```

Assembly: FITB Solution

S11 Q3

```
typedef struct node {
     void *data;
     struct node *next;
} node_t;

node_t *fun(node_t *n, int f(node_t *)) {
     node_t *a, *b;
     if (n == NULL) {
          return NULL;
     }

     a = fun(n->next, f);

     if (f(n) > 0) {
          b = malloc(16);
          b->data = n->data;
          b->next = a;
          return b;
     }

     return a;
}
```

```
114d: push   %r12
114f: push   %rbp
1150: push   %rbx
1151: mov %rdi,%rbx
1154: test   %rdi,%rdi
1157: je  1192 <fun+0x49>
1159: mov %rsi,%rbp
115c: mov 0x8(%rdi),%rdi
1160: call   1149 <fun>
1165: mov %rax,%r12
1168: mov %rbx,%rdi
116b: call   *%rbp
116d: test   %eax,%eax
116f: jg  1179 <fun+0x30>
1171: mov %r12,%rax
1174: pop %rbx
1175: pop %rbp
1176: pop %r12
1178: ret
1179: mov $0x10,%edi
117e: call   1050 <malloc@plt>
1183: mov (%rbx),%rdx
1186: mov %rdx,(%rax)
1189: mov %r12,0x8(%rax)
118d: mov %rax,%r12
1190: jmp 1171 <fun+0x28>
1192: mov %rdi,%r12
1195: jmp 1171 <fun+0x28>
```

Assembly: Reasoning over Code

```
f:
        leal    1(%rdi), %ecx
        movl    $0, %edx
.L5:
        leal    -1(%rcx), %eax
        cmpl    %edx, %eax
        je      .L8
        leal    (%rdx,%rcx), %eax
        shrl    %eax
        movl    %eax, %esi
        imull   %eax, %esi
        cmpl    %esi, %edi
        jb      .L9
        movl    %eax, %edx
        jmp     .L5
.L9:
        movl    %eax, %ecx
        jmp     .L5
.L8:
        movl    %edx, %eax
        ret
```

- You are given the assembly for a mystery function **`f`**.
- Assume that **`f`** takes in a single unsigned integer **`y`** as its argument.
- For what *range* of inputs does **`f`** return **`8`**?

Solution

- We might translate the assembly code to C code as follows:

```
c = y + 1;
d = 0;

while (d != c - 1) {
    a = (c + d) / 2;

    if (y < a * a) {
        c = a;
    } else {
        d = a;
    }
}

return d;
```

- This computes the integer square root of a number, so the correct range is 64-80.

Source: Wikipedia

Shells & File IO

Shells & File IO

- Common exam questions:
  - Code Tracing
- Make sure you understand the syscalls!
  - wait, waitpid
  - fork
  - execve
  - dup2

Shells & File IO

```
int main() {
    char c;
    int fd1 = open("input1.txt", O_RDWR);
    int fd2 = open("input1.txt", O_RDWR);
    int fd3 = open("output.txt", O_WRONLY
              |O_CREAT|O_TRUNC, DEF_MODE);

    read(fd1, &c, 1);
    int pid = fork();
    if (pid == 0) {
        rw1(fd2, fd3);
        rw1(fd1, fd3);
        int pid2 = fork();
        if (pid2 == 0) {
            rw1(fd1, fd3);
        }
        // else wait(NULL);
        rw1(fd2, fd3);
    }
    else {
        wait(NULL);
        rw1(fd2, fd3);
        dup2(fd2, fd1);
        rw1(fd2, fd3);
    }
    close(fd1);
    close(fd2);
    close(fd3);
}
```

```
// input1.txt contains "MAKOSHARK"

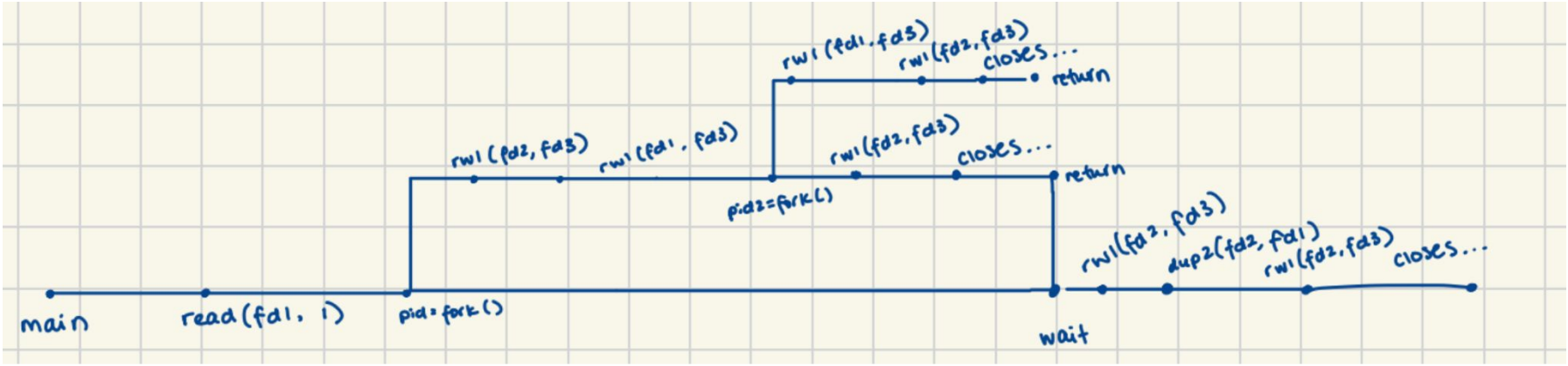
// Read one char and write one char
static void rw1(int in, int out) {
    char c;
    read(in, &c, 1);
    write(out, &c, 1);
}
```

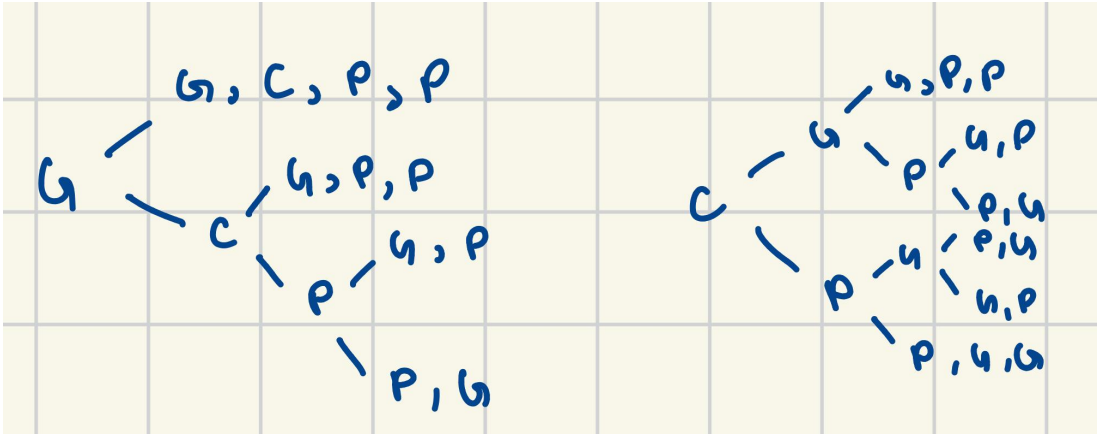
- How many possible process orderings are there?
- If we uncomment the **`wait`** call, what's the output?

Shells & File IO

- How many possible orderings of operations are there?

  10 versions

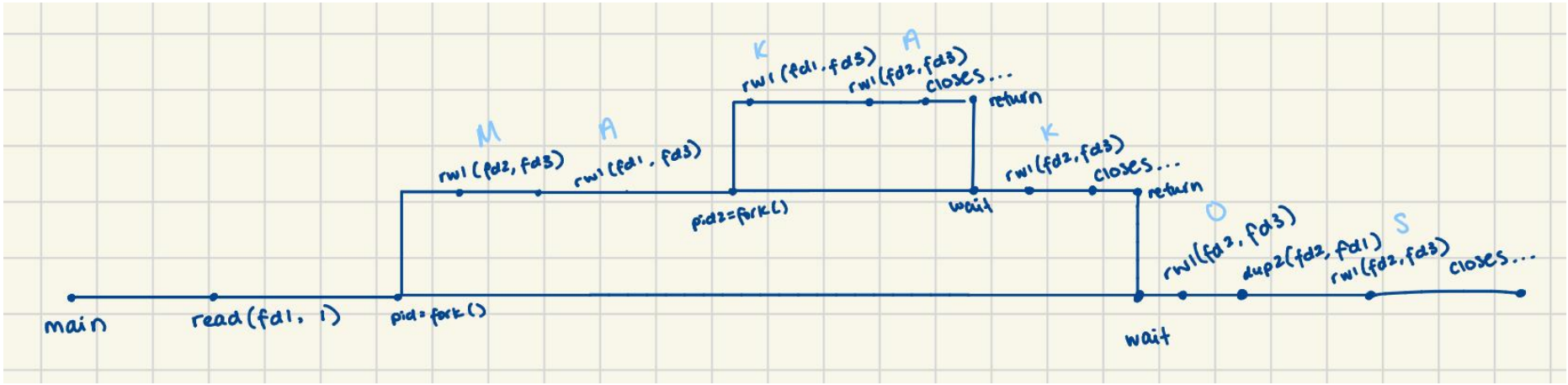




Shells & File IO

- If we uncomment the **`wait`** call, what's the output?

  MAKAKOS



Synchronization & Threading

Synchronization & Threading

- Common exam questions:
  - Making code thread-safe
  - Removing deadlocks/data races from in-progress parallel implementations
- Make sure you understand:
  - Coarse-grained vs Fine-grained implementations
  - What is a deadlock?
  - What is a data race?
  - What is a critical section?

Synchronization & Threading

```
// requires that root is not NULL
void insert (
     struct node *root,
     int val
)
{
     if (!root->next) {
          root->next = calloc(
               sizeof(struct node), 1
          );
          root->next.val = val;
     } else (
          insert(root->next, val);
     }
}
```

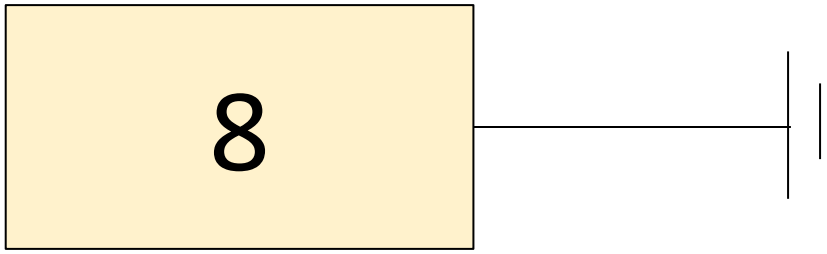
```
// Linked list implementation

struct node {
     int data;
     struct node *next;
}
```

- When insert is called in parallel, we notice potential lost inserts. Why?

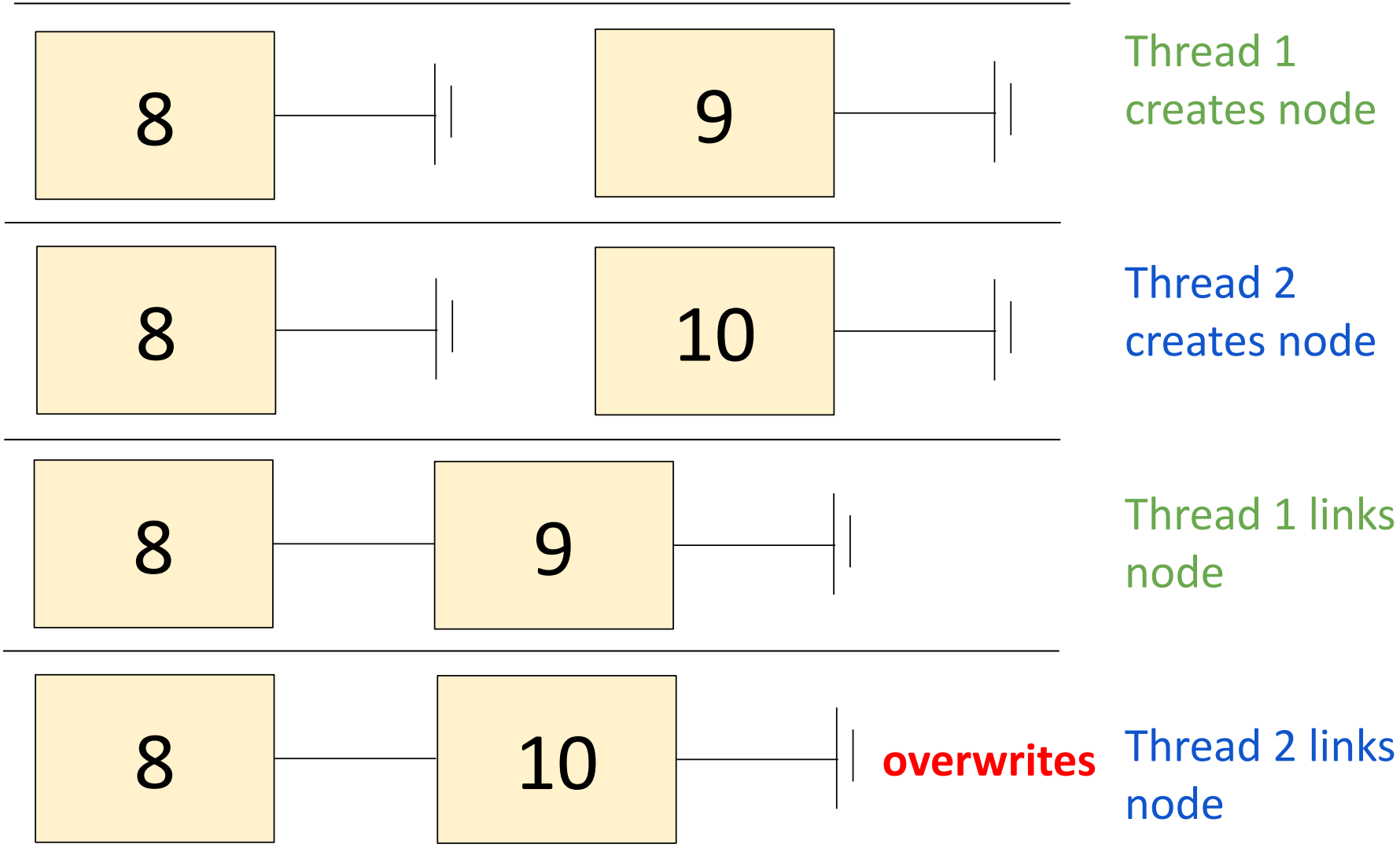
Synchronization & Threading

We call **insert(9)** and **insert(10)** in parallel on the following linked list.



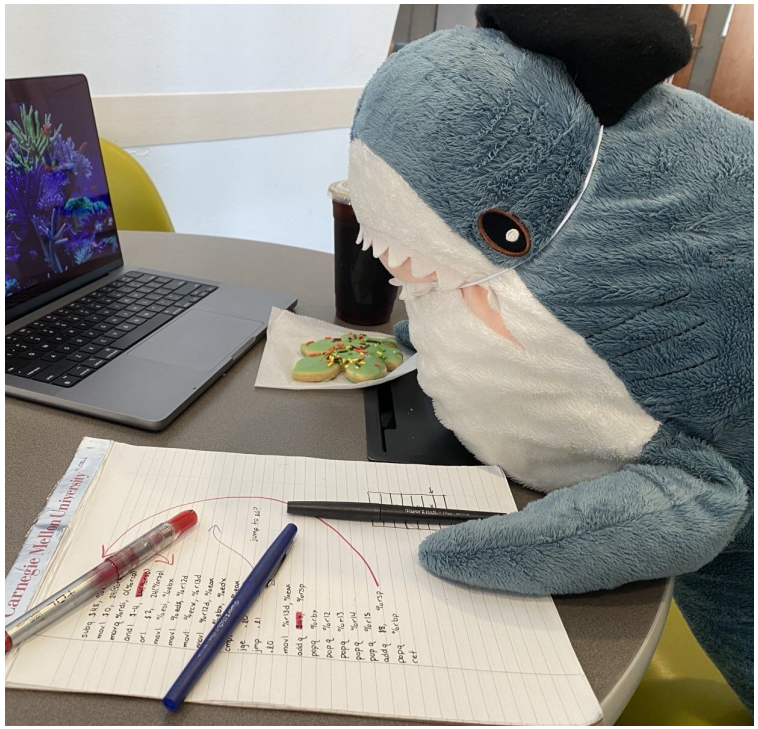
Thread 1 calls **insert(9)**.
Thread 2 calls **insert(10)**.
Potential data race trace…

Synchronization & Threading



Wrapping Up

- ***Final Exam Bootcamp***
  - Saturday April 25, 11-2
  - GHC 4401
- ***Final Exam***
  - Monday April 27th, 1-4PM
  - Watch Ed for details!
- Good luck for Finals Week :-)



The End

Memory Layout

Memory Layout

Here is an array with 6 elements:

```
short arr[] = {0x1234, 0x8326, 0x9742, 0x4200, 0x1521, 0x3531};
```

Given the array's starting address is **0x7fffffffdf86**, what do these statements print? Justify your answers.

```
printf("2: %lx\n", *((long*)&arr[2]));
printf("3: %x\n", (unsigned char) *((char*)&arr[3]));
printf("4: %x\n", (unsigned char) *(((char*)arr)+ 3));
printf("5: %x\n", ((int*)arr)[1]);
```

Answers

```
printf("2: %lx\n", *((long*)&arr[2]));

printf("3: %x\n", (unsigned char)*((char*)&arr[3]));

printf("4: %x\n", (unsigned char) *(((char*)arr)+ 3));

printf("5: %x\n", ((int*)arr)[1]);
```

2: 353115214200974 2

3: 0

4: 83

5: 42009742